

A directory, and who is allowed to read it

OpenLDAP populated from empty, queried from another machine, and asked the same question twice — once anonymously and once authenticated — to show that the answer depends on who is asking. Captured from a running pair.

Command `qlab run ldap-lab`

VMs `ldap-lab-server 192.168.100.1 · ldap-lab-client 192.168.100.2`

Directory `dc=ldap-lab,dc=local`

Outcome `qlab test ldap-lab` → 6 exercises, 28 checks, all passed

1. Two machines, and an empty directory

```
--- server ---
ens3          UP          10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4          UP          192.168.100.1/24 fe80::5054:ff:fe00:401/64
--- client ---
ens3          UP          10.0.2.15/24 metric 100 fec0::5054:ff:fe12:3456/64
fe80::5054:ff:fe12:3456/64
ens4          UP          192.168.100.2/24 fe80::5054:ff:fe00:402/64
```

active

```
$ what the directory holds before exercise 1
dn:
namingContexts: dc=ldap-lab,dc=local
```

slapd is running, but `namingContexts` says `dc=nodomain` — the Debian default for a package installed without being configured. There is a server and no directory in it. Exercise 1 of the guide fixes that with a script:

```
$ bash ~/demo-setup.sh
Admin DN:      cn=admin,dc=ldap-lab,dc=local
Admin password: admin

Users: alice (alice123), bob (bob123), charlie (charlie123)
Group: developers (alice, bob, charlie)

phpLDAPadmin: run 'qlab ports' on the host to see the URL
(from host browser, login with admin DN above)

Try from the client VM:
ldapsearch -x -H ldap://192.168.100.1 -b "dc=ldap-lab,dc=local" "(uid=alice)"
```

2. A directory is a tree, and the path is the name

```
$ the whole directory, as a tree of distinguished names
dn: dc=ldap-lab,dc=local

dn: ou=users,dc=ldap-lab,dc=local
dn: ou=groups,dc=ldap-lab,dc=local

dn: uid=alice,ou=users,dc=ldap-lab,dc=local
dn: uid=bob,ou=users,dc=ldap-lab,dc=local
dn: uid=charlie,ou=users,dc=ldap-lab,dc=local
dn: cn=developers,ou=groups,dc=ldap-lab,dc=local
```

Every entry's name — its **distinguished name** — is its full path through the tree, read from the leaf upwards. `uid=alice,ou=users,dc=ldap-lab,dc=local` is both the identifier and the location;

there is no separate key. Move an entry and its name changes, which is why renaming in LDAP is a different operation from editing.

The root here is `dc=ldap-lab,dc=local`, built from *domain components* — an old convention that maps a DNS name onto a tree root. `ou` is an organisational unit, which is nothing more than a container you chose to create.

3. An entry is a set of attributes, and its classes decide which

```
$ one entry in full
dn: uid=alice,ou=users,dc=ldap-lab,dc=local
objectClass: inetOrgPerson
objectClass: posixAccount
objectClass: shadowAccount
uid: alice
sn: Smith
givenName: Alice
cn: Alice Smith
displayName: Alice Smith
uidNumber: 10001
gidNumber: 10000
homeDirectory: /home/alice
loginShell: /bin/bash
mail: alice@ldap-lab.local
```

Read the `objectClass` lines first, because they are the schema: they declare what this entry *is*, and between them they decide which attributes are required and which are merely allowed.

- `inetOrgPerson` brings the human fields — `cn`, `sn`, `givenName`, `mail`.
- `posixAccount` brings the Unix ones — `uidNumber`, `gidNumber`, `homeDirectory`, `loginShell`.

That second class is the whole reason LDAP is used for login: those are exactly the fields `/etc/passwd` holds. A machine configured to consult this directory gets its users from here instead of from a local file, and `alice` becomes the same account everywhere.

Note also that an attribute may appear more than once — `objectClass` does here. LDAP attributes are multi-valued by nature, which is unlike a database column.

```
$ a group, and how membership is expressed
dn: cn=developers,ou=groups,dc=ldap-lab,dc=local
objectClass: posixGroup
cn: developers
gidNumber: 10000
memberUid: alice
memberUid: bob
memberUid: charlie
```

Membership is `memberUid`, listed on the **group**, not on the users. So "which groups is alice in" is a search across groups, not a field you read on alice. That is the opposite of the intuition most people bring, and it is why `memberOf` exists as an optional overlay that maintains the reverse pointer.

4. Filters

```
$ (uid=*)                                every entry that has a uid
uid: alice
uid: bob
uid: charlie

$ (&(objectClass=posixAccount)(uidNumber>=1002))    AND, with a range
uid: alice
uidNumber: 10001
uid: bob
uidNumber: 10002
uid: charlie
uidNumber: 10003

$ (|(uid=alice)(uid=bob))                OR
uid: alice
uid: bob
```

LDAP filters are prefix notation: the operator comes first, then its operands, each in its own parentheses. `(&(a)(b))` is AND, `(|(a)(b))` is OR, `(!(a))` is NOT. It reads badly at first and composes well once you accept it.

`(uid=*)` is a presence test — entries that have the attribute at all, regardless of value. It is also the cheapest useful filter for finding out what kind of entries a directory contains.

5. Over the network, and who is asking

```
$ the same query, from the other machine over the network
dn: uid=alice,ou=users,dc=ldap-lab,dc=local
cn: Alice Smith
mail: alice@ldap-lab.local
```

The same query from the other machine. Nothing is installed on the client beyond `ldapsearch`; a directory is a network service, and this is the normal way to use it.

```
$ anonymous bind (-x, no credentials)
dn: uid=alice,ou=users,dc=ldap-lab,dc=local

$ authenticated bind as the directory manager
dn: uid=alice,ou=users,dc=ldap-lab,dc=local
userPassword:: e1NTSEF9THA2NFRiZ204WDU3R1grZWsl a3B4dno30Tg3ZFQwQW0=
```

Now the point. Both queries ask for `userPassword` on the same entry:

- **anonymously**, the entry comes back with the dn and nothing else — the attribute was withheld, without an error;
- **bound as the directory manager**, the attribute is there.

That is LDAP access control working as intended, and the shape of it is worth noticing: you are not told that something was hidden. An anonymous search that returns fewer attributes than you expected is the normal signal that you should have authenticated.

The value itself is base64 (`::` rather than `:` after the attribute name) and inside it is `{SSHA}` — a salted hash, not the password.

```
directory reset for the exercises.  
dn:  
namingContexts: dc=ldap-lab,dc=local
```

6. Verification

```
Exercise 1: All 7 checks passed  
Exercise 2: All 6 checks passed  
Exercise 3: All 3 checks passed  
Exercise 4: All 5 checks passed  
Exercise 5: All 3 checks passed  
Exercise 6: All 4 checks passed  
Exercises run:      6  
Exercises passed:   6  
Exercises failed:   0  
Exercises skipped:  0  
All exercises passed!
```

7. What to take away

- The distinguished name is the path. Identity and location are the same thing.
- `objectClass` is the schema: it decides which attributes an entry may and must have.
- `posixAccount` is what makes a directory usable for Unix login.
- Membership lives on the group as `memberUid`; the reverse view needs an overlay.
- Filters are prefix notation. `(uid=*)` tests presence.
- An anonymous bind is a real identity with real limits, and being denied an attribute looks like the attribute not existing.

`guide.md` in the plugin carries the exercises: configuring the domain by hand, building the tree, adding users and groups, querying from the client, phpLDAPAdmin, and modifying entries.